

Claudia Team Collaboration: Brainstorm & Critique

Date: April 1, 2026

Status: Brainstorm / Early Exploration

Author: Kalin Ovtcharov

Executive Summary

Claudia is currently a **single-user, multi-agent orchestrator** — one developer spawning and managing multiple Claude Code tasks across workspaces. This document explores how to evolve Claudia into a **team-aware platform** where multiple developers can coordinate AI-assisted work, share context, and avoid stepping on each other's toes.

Each idea is presented with an honest critique assessing feasibility, gaps, and whether it genuinely adds value or just adds complexity.

Current Architecture Constraints

Before diving into ideas, it's worth noting what we're building on:

- **No database** — all state lives in JSON files (`tasks.json`, `workspace-config.json`, `learnings.json`)
- **No user identity** — the system has no concept of "who" is connected
- **Single-process server** — one Express + WebSocket server, one set of PTY processes
- **Local-first** — everything runs on the developer's machine
- **40+ WebSocket message types** — real-time communication is already mature

These constraints shape what's easy (adding fields to existing models) vs. what requires architectural shifts (centralized deployment, database migration).

Idea 1: Shared Task Visibility

Concept

Let team members see each other's active tasks and their states in real-time. Add a `userId` / `userName` field to the Task model. The sidebar groups tasks by developer.

Critique

- **Dependency:** Meaningless without authentication (Idea 2). You can't show "who" is doing what if you don't know who anyone is.
- **Privacy concern:** Not all tasks should be visible. A dev experimenting with a hacky approach may not want the team watching.
- **Limited standalone value:** Seeing task names without context (the conversation, the diff) is only marginally useful.
- **Verdict:** Bundle this into the auth implementation rather than treating it as a standalone feature. It's a UI enhancement on top of identity, not a feature in itself.

Refined Recommendation

Ship this as part of Phase 1 (auth), not as its own deliverable. Add a **visibility** toggle per task: **private** (default) or **team-visible**.

Idea 2: Authentication & User Identity

Concept

Add auth so the server knows who is connected. Options range from simple tokens to GitHub OAuth to enterprise SSO.

Critique

- **Essential foundation** — nothing else works without this. Correctly identified as the prerequisite.
- **Token-per-user is too fragile** for real teams. Tokens get shared in Slack, leaked in screenshots, never rotated. Skip this for anything beyond a demo.
- **GitHub OAuth is the sweet spot** — devs already have accounts, it ties identity to repo permissions, and the implementation is well-documented. But it requires a callback URL, which complicates local-first deployment.
- **Session management complexity** — currently the WebSocket has no auth handshake. Adding one means handling reconnection auth, token refresh, and session expiry. This touches **useWebSocket.ts** deeply.
- **Risk:** Over-engineering auth for a tool that might stay small-team. Don't build an enterprise IAM system.

Refined Recommendation

Start with **GitHub OAuth** for teams that deploy Claudia on a shared server. For local-first usage, use a simpler **shared secret / passphrase** model where the server operator sets a team password. Avoid building a full user management system — lean on GitHub for identity and avatars.

Idea 3: Shared vs. Private Workspaces

Concept

Workspaces can be **private** (only creator sees them) or **shared** (team-visible). Add **visibility** and **members[]** to the Workspace model.

Critique

- **ACL model is underspecified.** What permissions do members get? Read-only observation? Full task creation? Can they kill another dev's task? Without answering this, you'll build the wrong thing.
- **Workspace = directory path** in the current model (**id** is literally the filesystem path). Sharing a "workspace" means multiple devs need access to the same directory, which implies a shared filesystem or a centralized server. This is a bigger assumption than it looks.
- **Good mental model** — developers intuitively understand "shared project" vs. "my sandbox." The UX is natural.
- **Missing:** Workspace-level roles (owner, contributor, observer) would make this much more useful.

Refined Recommendation

Implement with three permission levels: **owner** (full control), **contributor** (can create/manage their own tasks), **observer** (read-only view of tasks and terminal output). Default new members to observer.

Idea 4: Task Handoff & Collaboration

Concept

Devs can assign, watch, or take over each other's tasks. Add **assignedTo**, **watchers[]** fields.

Critique

- **"Takeover" is technically hard.** Claude Code sessions are tied to PTY processes with local state. You can't just transfer a running PTY to another user's context. The session would need to be serialized or the new user would need to connect to the same PTY stream.
- **Watch mode is easy and high-value.** You already stream **task:output** via WebSocket — routing it to additional connections is straightforward. This should ship first.
- **Assignment is socially awkward.** In practice, devs don't "assign" AI tasks to each other — they discuss in Slack and each person runs their own tasks. The real need is **context sharing**, not task reassignment.
- **Better framing:** Instead of "handoff," think "continuation." Dev A's task produces a diff and conversation summary. Dev B can start a *new* task that includes that context as a preamble.

Refined Recommendation

Build **watch mode** first (low effort, high value). Replace "takeover" with **task context export** — generate a shareable summary (prompt + key decisions + resulting diff) that another dev can use to start an informed follow-up task.

Idea 5: Activity Feed / Team Dashboard

Concept

A shared feed showing task creation, completion, state changes, and file modifications across the team.

Critique

- **Duplicates existing tools.** Most teams already have Slack, Teams, or Discord for async awareness. Building another notification surface inside Claudia competes with where devs already look.
- **Noise risk.** A busy team of 5 devs running 3-4 tasks each generates dozens of state changes per hour. Without aggressive filtering, the feed becomes noise.
- **Better as an integration.** Send key events (task completed, conflict detected, review needed) to Slack/Teams via webhooks rather than building a custom feed UI.
- **Dashboard has value** if it's a bird's-eye view (who's working on what, which workspaces are active), not a chronological log.

Refined Recommendation

Build a **team overview dashboard** (current state, not history) and add **webhook/Slack integration** for notifications rather than an in-app activity feed. Let teams use their existing communication tools.

Idea 6: Conflict Detection & Git Coordination

Concept

Warn when two tasks modify overlapping files or work on the same branch. Cross-reference `gitState.filesModified` across users.

Critique

- **Timing problem.** `filesModified` is populated *after* changes are made. By the time you detect the conflict, both devs have already done the work. Real conflict prevention needs to happen *before* or *during* file writes.

- **Branch-level vs. file-level.** Two devs on the same branch is a hard conflict. Two devs modifying the same file on different branches is a soft conflict (merge may resolve it). The system needs to distinguish these.
- **High value despite limitations.** Even after-the-fact detection saves time. "Heads up, Sarah also modified `auth.ts` in the last hour" is useful even if it's not preventive.
- **Missing: branch coordination.** Auto-creating feature branches per task (which Claudia could do) would eliminate most conflicts by default.

Refined Recommendation

Implement in two layers: (1) **Proactive** — auto-create a feature branch per task so work is isolated by default. (2) **Reactive** — detect overlapping file modifications and notify both devs via the team dashboard and webhook. Don't try to prevent conflicts; surface them early.

Idea 7: Shared Learnings & Knowledge Base

Concept

Make the existing learnings system (`learnings.json` with vector embeddings and MemRL scoring) team-shared.

Critique

- **Quality control problem.** One dev's learnings might be wrong, outdated, or specific to their workflow. Without curation, the shared knowledge base becomes a junk drawer.
- **MemRL utility scoring helps.** The existing `utility`, `useCount`, and `successCount` fields provide natural quality signals. Learnings that help multiple devs will score higher.
- **Storage migration needed.** JSON files don't support concurrent writes from multiple users. Need to move to SQLite at minimum for shared learnings.
- **Privacy concern.** Some learnings may contain sensitive context (credentials, internal URLs, proprietary logic). Need a way to mark learnings as private.
- **High long-term value.** This is where team collaboration with AI gets genuinely novel — institutional knowledge that compounds across the team.

Refined Recommendation

Move learnings to **SQLite** with per-learning `visibility` (private/team). Add a **curation step** — learnings start as private and can be explicitly "published" to the team. Use MemRL scoring to surface high-value shared learnings.

Idea 8: Centralized Server Deployment

Concept

Deploy one shared Claudia server instead of each dev running their own instance. Replace JSON storage with a real database.

Critique

- **Security is the elephant in the room.** Claudia spawns shell processes (PTY) that execute arbitrary code. On a shared server, one dev's task could access another dev's files, environment variables, or credentials. Process isolation (containers, VMs) becomes mandatory.
- **Resource contention.** Claude Code tasks are CPU/memory intensive. Multiple devs running concurrent tasks on one server need resource limits, queuing, and fair scheduling.
- **The hybrid model is more practical.** Keep task execution local (each dev runs PTY processes on their own machine) and sync metadata/state through a lightweight coordination server. This avoids the security and resource problems.
- **Database migration is straightforward.** The current store abstractions ([WorkspaceStore](#), [ConfigStore](#), [LearningsStore](#)) have clean interfaces. Swapping JSON for SQLite/Postgres is a bounded refactor.

Refined Recommendation

Go **hybrid**: a lightweight coordination server (auth, shared state, learnings, notifications) with task execution remaining local. Each dev's Claudia instance connects to the coordination server for team features while running Claude Code locally. This preserves security, avoids resource contention, and is incrementally adoptable.

Idea 9: Review & Approval Workflows

Concept

Before a task's changes are committed/pushed, another team member can review within Claudia.

Critique

- **Reinventing GitHub PRs.** Code review is a solved problem with deep tooling (GitHub, GitLab, Bitbucket). Building a parallel review system inside Claudia fragments the workflow and loses integration with CI, status checks, and existing review norms.
- **The actual gap is pre-PR review.** The interesting moment isn't "review this PR" — it's "should this task's approach be committed at all?" That's a lighter-weight check that GitHub doesn't cover.

- **Better as integration.** Auto-create a draft PR from a completed task, with the conversation summary as the PR description. Let review happen where it normally does.

Refined Recommendation

Don't build a review system. Instead, add a **one-click "Create Draft PR"** action on completed tasks that auto-generates the PR with a summary derived from the conversation history. Keep review in GitHub.

Idea 10: Supervisor Chat as Team Channel

Concept

Make the existing SupervisorChat team-aware — a shared AI supervisor that sees all team members' tasks.

Critique

- **Context explosion.** The supervisor currently has tools to manage tasks (`create_task`, `list_tasks`, etc.). With a full team's tasks, the supervisor's context window fills up fast. It needs smart filtering to remain useful.
- **Conflicting instructions.** If two devs ask the supervisor contradictory things ("prioritize the auth refactor" vs. "hold off on auth"), who wins? Need clear ownership semantics.
- **Good for coordination queries.** "Is anyone working on the payment module?" or "What's the status across the team?" are genuinely useful queries that no other tool answers as naturally.
- **Noisy in practice.** A shared chat with AI responses interleaved from multiple users gets confusing fast.

Refined Recommendation

Keep per-user supervisor chat (private) but add a **team query mode** where the supervisor can read (but not modify) other users' task states. This enables "who's working on X?" queries without the chaos of a shared chat. Add a `/team-status` command that generates a snapshot summary.

Ideas Not Originally Considered

The original brainstorm missed several important dimensions:

11. Integration with Existing Team Tools

Slack/Teams webhooks for task notifications, **Linear/Jira integration** for linking tasks to tickets, **GitHub Issues** for auto-linking tasks to issues. Teams already have workflows — Claudia should plug into them, not replace them.

12. API Rate Limiting & Cost Management

Multiple devs hitting Claude API simultaneously could blow through usage limits or budgets. Need per-user or per-team **usage quotas**, a **cost dashboard**, and the ability to set spending limits.

13. Task Templates & Playbooks

Teams develop repeatable patterns ("run the migration checker before merging," "always lint + test after refactoring"). **Shared task templates** let the team codify best practices as reusable prompts with pre-configured workspace, system prompt, and follow-up steps.

14. Audit Trail & Compliance

For regulated industries or security-conscious teams, a log of "who ran what, when, with what permissions, and what changed" is essential. The existing `gitState` tracking is a start but needs to be formalized as an append-only audit log.

15. Role-Based Access Control (RBAC)

Beyond workspace-level permissions, teams need org-level roles: **admin** (manage server config, users), **developer** (full task access), **viewer** (read-only dashboards). This matters for security teams, managers, or stakeholders who want visibility without execution access.

Prioritized Roadmap

Based on the critiques above, here's a revised and more honest roadmap:

Phase 1: Identity & Awareness (Foundation)

Feature	Effort	Notes
GitHub OAuth	Medium	Foundation for everything else
User model + task ownership	Low	Add <code>userId</code> to Task, filter by user
Shared task visibility (opt-in)	Low	Per-task <code>visibility</code> toggle
Team overview dashboard	Medium	Who's doing what, right now

Key risk: OAuth callback URL complicates local-first deployment. Mitigate with a simpler shared-secret fallback.

Phase 2: Coordination & Safety

Feature	Effort	Notes
Auto feature branches per task	Low	Prevents most git conflicts
File overlap detection + alerts	Medium	Reactive conflict warnings
Slack/webhook notifications	Medium	Use existing team channels
One-click draft PR creation	Medium	Bridges Claudia to GitHub workflow

Key risk: Conflict detection timing — alerts after the fact are useful but not preventive.

Phase 3: Shared Knowledge & Templates

Feature	Effort	Notes
SQLite migration for learnings	Medium	Required for concurrent access
Shared learnings with curation	Medium	Publish/private toggle, MemRL scoring
Task templates / playbooks	Medium	Reusable team-defined patterns
Team supervisor queries	Low	Read-only cross-user task awareness

Key risk: Learning quality — need curation mechanisms to prevent noise.

Phase 4: Infrastructure & Scale

Feature	Effort	Notes
Hybrid coordination server	High	Central state sync, local execution
Usage quotas & cost dashboard	Medium	Per-user API budget tracking
RBAC (admin/dev/viewer)	Medium	Org-level permission tiers
Audit trail	Medium	Append-only log of actions

Key risk: Hybrid architecture complexity — needs careful design to avoid becoming a distributed systems problem.

Architectural Decision: Local-First vs. Centralized

The single biggest decision is the deployment model:

	Local-First + Coordination Server	Fully Centralized
Security	Tasks run locally, no cross-user risk	Needs container isolation per user
Setup	Each dev installs Claudia + connects to coord server	One server, devs connect via browser
Resource mgmt	Each dev uses their own machine	Shared server needs scheduling/limits
Offline work	Works offline, syncs when connected	Requires server connection
Complexity	Medium (sync protocol)	High (isolation, scheduling, storage)
Best for	Small-medium teams (2-15 devs)	Large orgs with infra teams

Recommendation: Start with local-first + coordination server. It's incrementally adoptable (devs can opt in without changing their workflow), avoids the hardest security problems, and matches how dev tools typically evolve.

Summary of Key Insights from Critique

1. **Don't rebuild what exists.** Review in GitHub, notifications in Slack, tickets in Linear. Claudia should integrate, not compete.
2. **Identity is the foundation.** Nothing works without knowing who's who. Ship auth first.
3. **Hybrid beats centralized.** Local execution with shared coordination avoids the hardest infrastructure problems.
4. **Conflict prevention > conflict detection.** Auto-creating feature branches eliminates most conflicts before they happen.
5. **Shared knowledge is the unique value.** Team-wide learnings that compound over time is something no other tool offers. Invest here.
6. **Start with watch mode.** Letting devs observe each other's tasks is low-effort, high-trust, and immediately useful.